

LitNotes Canvas — UX & System Design

Tablet-first legal research app · "Obsidian Chamber" theme · Supabase cloud accounts + Figma-style live sharing.

Updated Jul 14, 2026 · demo target Aug 9

Where it stands

The full research app is built and runs 100% offline today. Cloud sign-in (Supabase + Google) and live workspace sharing are implemented end-to-end, type-check clean, and switch on the moment you add Supabase keys — no code changes. They have not yet been run on a device, so a short dry-run is the main thing between here and the demo.

Status at a glance

14

Shipping (offline)

5

Coded · needs
keys/device

6

Pending / not built

0

TypeScript errors

What's built & how good it is

Shipping capabilities that work on-device today. Quality is a candid read, not a checkbox.

CAPABILITY	AREA	QUALITY
PDF reader — open, paginate, per-page OCR + scan-all	Reader	Solid
Highlight → categorise → excerpt card	Annotation	Solid
Infinite canvas — pan/zoom, drag, edges, ink, links	Canvas	Solid
Neon thread overlay (Skia glow)	Canvas	Solid
AI research — offline pack + live memo → card	AI	Good
Search across highlights / notes / OCR text	Organise	Good
Bundle index / bookmarks with reorder	Organise	Good
Export — canvas PNG + notes Markdown	Output	Good
Obsidian Chamber theme — light/dark/system, reactive	Design	Solid
App flow — splash → auth → perms → onboarding → workspace	Shell	Solid
Local offline accounts + persisted session	Auth	Good
Settings — 8 sections incl. Reading & Collaboration	Settings	Solid
Per-user workspace isolation (no cross-account leak)	Data	Solid
Auto-OCR toggle + persisted permission state	Settings	Solid

Built but not yet verified

Code complete and compiling; each needs credentials and/or an on-device pass before it is demo-safe.

CAPABILITY	STATE	WHAT IT NEEDS
Cloud auth — Supabase email + Google sign-in	Coded, untested	Supabase keys + one native rebuild
Live sharing — presence, cursors, snapshot sync, roles	Coded, untested	Supabase keys; on-device dry-run
iOS permission prompts	Partial	Android real; iOS auto-granted (no native module)
Apple sign-in	Stub	Placeholder button only
Session lock on background	Design only	State modelled, not yet wired

Pending / not built

NOT BUILT YET	NOTES
PDF binary sync to peers (Supabase Storage)	Peers sync notes layer, not the file yet
CRDT per-field merge for simultaneous editing	Currently last-writer-wins snapshots
Automated tests / full QA sweep	No test suite yet
App icons, splash assets, store listing	Default RN assets
Crash / analytics opt-in	Not instrumented
Research offline queue + retries	Single-shot calls today

Quality assessment

AREA	RATING	ASSESSMENT
Core research loop (read → highlight → canvas → thread)	Strong	The product's spine. Fluid, tablet-first, Skia-smooth; the part most worth demoing.
Visual design & theming	Strong	Cohesive Obsidian Chamber palette, unified Lucide icons, reactive light/dark/system.
On-device data & accounts	Solid	Per-user isolation, debounced persistence, session restore. Local hashing is demo-grade, not production crypto.
AI research	Good	Works offline; live mode is genuinely useful with an Anthropic key. No queue/retry yet.
Cloud auth (Supabase + Google)	Promising	Cleanly abstracted with offline fallback; compiles and lints, but unverified on a device.
Live collaboration	Promising	Presence, cursors, roles and real-time canvas sync implemented end-to-end. Snapshot LWW (not CRDT); PDF file not yet shared; needs a multiplayer dry-run.
Testing & release hardening	Early	No automated tests, default app assets, no analytics. Expected for a pre-demo build.

Before the demo

1. Create a Supabase project, paste URL + anon key in Settings → Collaboration (no rebuild needed for that step).
2. (Optional) Configure Google OAuth and add the client IDs, then rebuild once for the new native Google Sign-In module.
3. Run a two-device dry-run of live sharing (presence, cursors, edit/view roles) and a light/dark + permissions pass.

Confirmed decisions

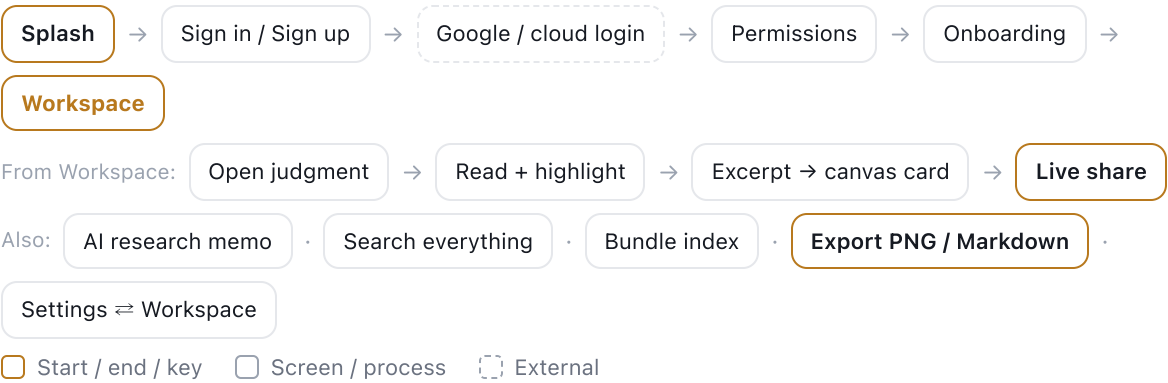
1. Theme follows the device default, defaults to Light when unset, with a manual System / Light / Dark override in Settings.
2. Settings covers Appearance, Reading, Account, AI & API key, Collaboration, Permissions, Data & backup, About.
3. Permissions (files, notifications, media for OCR) are requested during login and their grant-state is persisted.
4. Sharing backend is Supabase Realtime; identity is Supabase Auth with Google + email, and guests can join a live room from an invite link.

Theme tokens — Obsidian Chamber

TOKEN	DARK VALUE	ROLE
bg	#0A0C12	App base — blue-black
surface	#141822	Cards, toolbars
surfaceGlass	rgba(20,25,36,0.55)	Frosted panels
accent	#E8A13C	Primary — warm brass
ai	#38E0C8	AI / research channel
iris	#7C8CF8	Threads / links
text	#F0F2F7	Primary text

1 • User flow

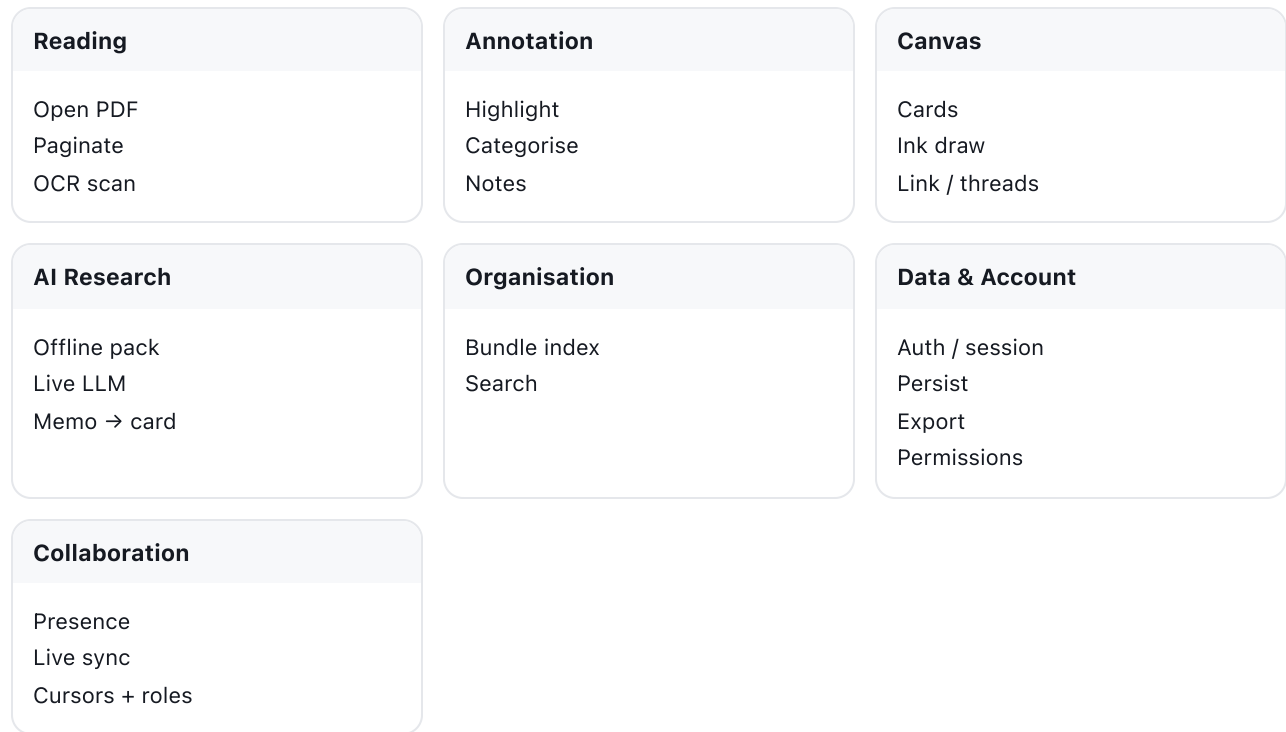
Splash → auth (local or cloud) → permissions → onboarding → workspace hub, then the core research loops and live sharing.



2 • Navigation / information architecture

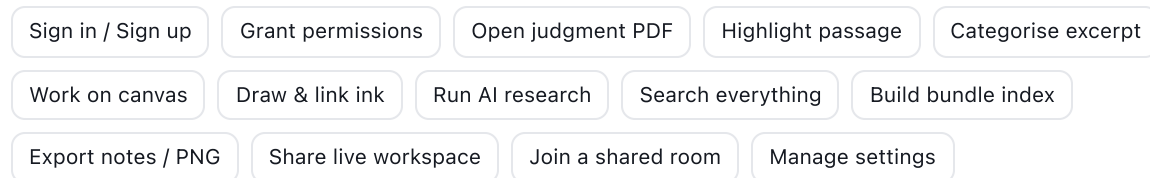
Auth	Workspace	Settings
Sign in / Sign up	Reader pane Canvas pane Side sheets: Search, Research, Index, Export, Share	Appearance Reading Account AI & API key Collaboration Permissions Data & backup About

3 · Functional decomposition

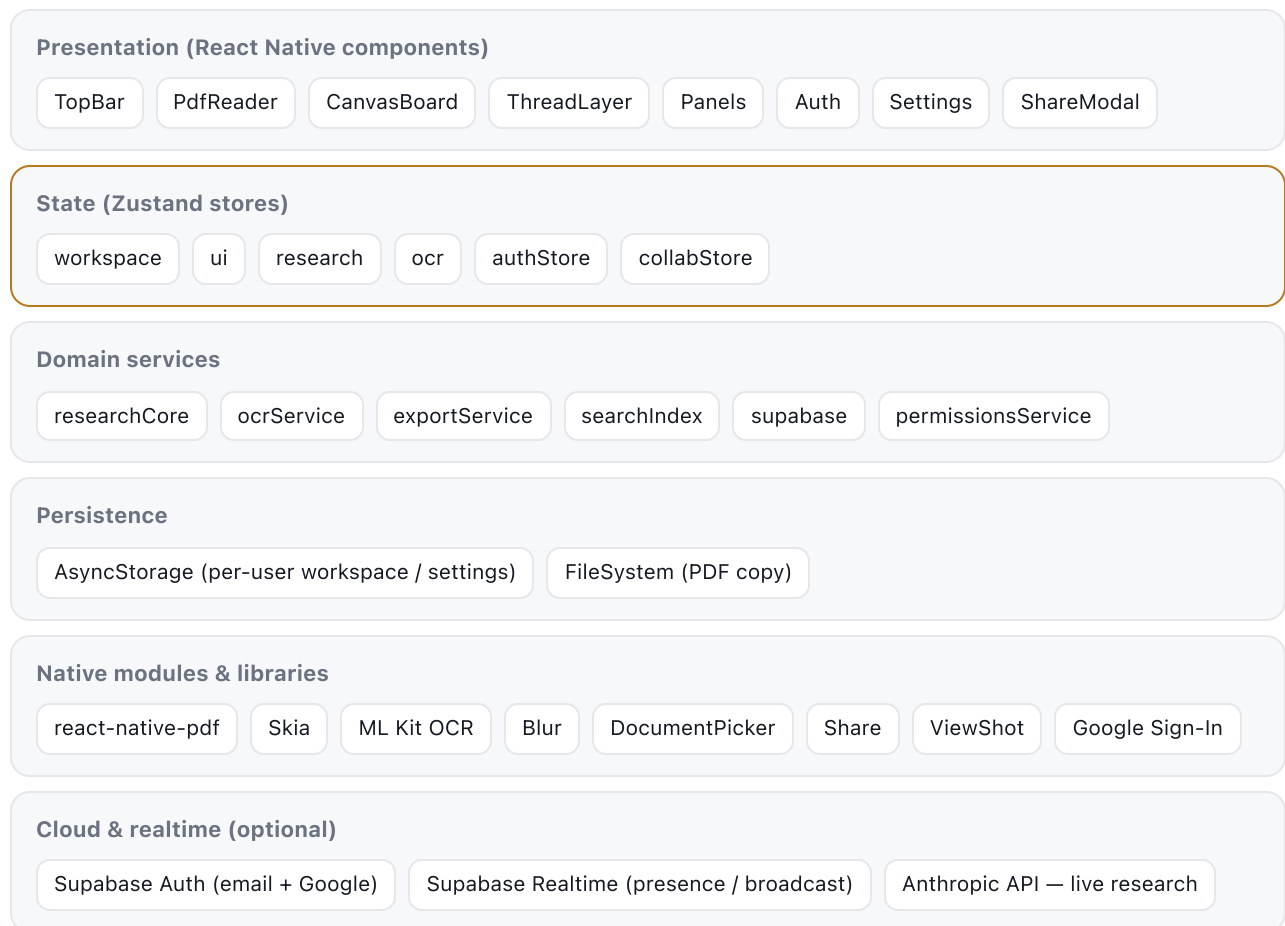


4 · Use case diagram

Primary actor: Legal Researcher. Secondary actors: Anthropic API, Supabase (auth + realtime).

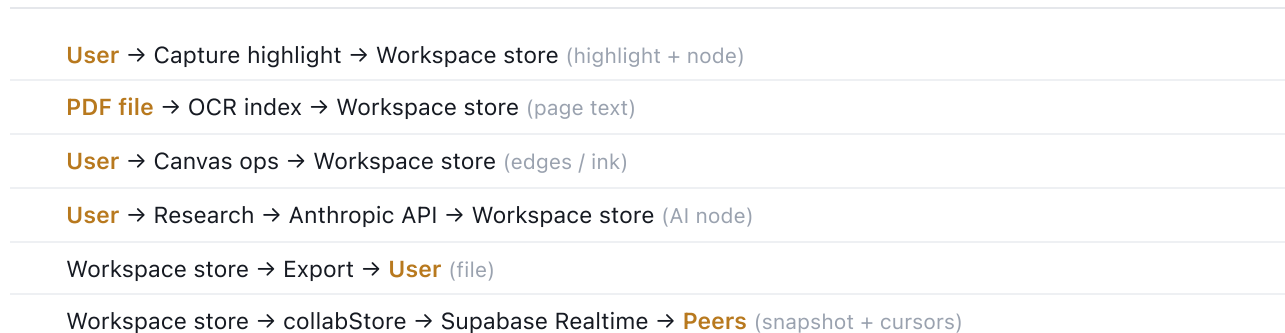


5 · System architecture (layered)



Data flows top → bottom; only the stores talk to persistence. The network is optional and opt-in: research, cloud auth and live sharing. With no keys, the app runs fully on-device.

6 · Data flow (level 1)



7 • Sequence — login & permissions

1. **User → SplashGate**: launch app
2. **SplashGate → Store**: check session
3. **Store → SplashGate**: *no session*
4. **SplashGate → AuthScreen**: show sign in
5. **User → AuthScreen**: submit credentials (or Google)
6. **AuthScreen → AuthService**: authenticate()
7. **AuthService → Store**: persist session + scope workspace
8. **AuthScreen → Permissions**: request files / notif / media
9. **User → Permissions**: grant / deny → saved
10. **AuthScreen → User**: *enter workspace*

8 • Sequence — highlight to canvas

1. **User → PdfReader**: tap + draw rect
2. **PdfReader → Popover**: open with rect
3. **User → Popover**: pick category + note
4. **Popover → Store**: addHighlight() + addExcerptNode()
5. **Store → Storage**: saveWorkspaceDebounce
6. **Store → CanvasBoard**: *re-render node → card appears*

9 • Live collaboration — how sharing works

A pluggable realtime layer on Supabase. The Share UI, presence, cursors and roles are identical regardless of backend; only the transport underneath is swappable.

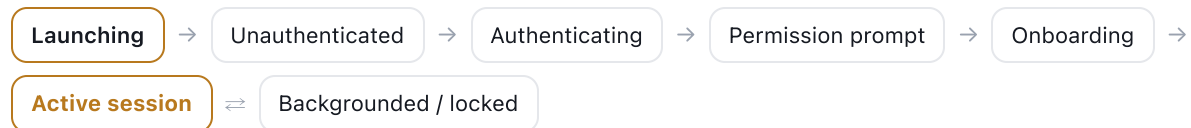
Transport	One Supabase Realtime channel per room (room:<CODE>). WebSocket, free tier.
Presence	Who is in the room + name, colour and role — drives avatar pips and the participant list.
State sync	Canvas, threads, ink, highlights, index and OCR broadcast as debounced snapshots (last-writer-wins); late joiners request the current state.
Cursors	Broadcast in board/world coordinates, rendered per-peer and inverse-scaled to stay crisp at any zoom.
Roles	Owner / editor / viewer. Viewers are read-only (canvas editing + PDF highlighting disabled).
Invites	litnotes://join/<CODE>?a=edit view deep link + 6-char room code; native share sheet.

10 · Sequence — live share session

1. **Owner → CollabStore**: startShare(edit|view)
2. **CollabStore → Realtime**: subscribe room:CODE + track presence
3. **CollabStore → Store**: getSyncState()
4. **CollabStore → Realtime**: broadcast snapshot
5. **Peer → Realtime**: join via link / code
6. **Realtime → Peer**: *presence sync (avatars)*
7. **Peer → Realtime**: request_snapshot
8. **Realtime → Peer**: *snapshot → applyRemote*
9. **Owner → Store**: edit card / ink
10. **Store → Realtime**: debounced snapshot
11. **Realtime → Peer**: *live update + cursor*

11 · Session state machine

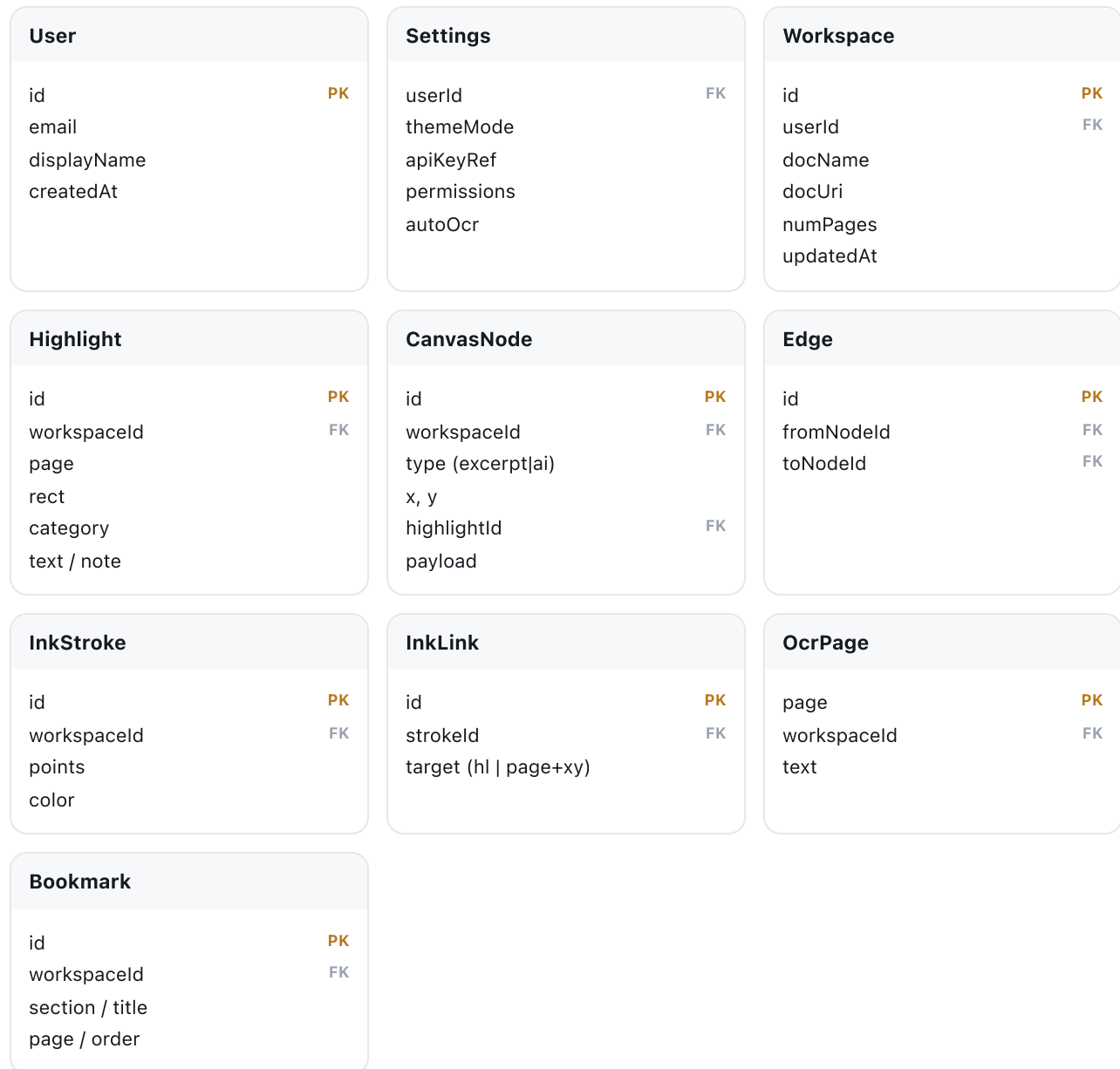
Dashed transitions are error / return / logout edges. "Backgrounded / locked" is modelled but not yet wired in code.



Authenticating → Unauthenticated on error · Active session → Unauthenticated on log out · Permission prompt → Active session for returning users.

12 · Data model (ER)

Workspaces are scoped per user; Settings persists theme, permissions and autoOcr. Live rooms are ephemeral (presence + broadcast), not persisted tables.



RELATIONSHIPS

FROM	CARDINALITY	TO
User	1 — 1	Settings
User	1 — *	Workspace
Workspace	1 — *	Highlight
Workspace	1 — *	CanvasNode
Highlight	1 — 0..1	CanvasNode

FROM	CARDINALITY	TO
CanvasNode	1 — *	Edge (from / to)
Workspace	1 — *	InkStroke → InkLink
Workspace	1 — *	OcrPage / Bookmark

13 · SDLC — where we are

1 · Requirements Done Legal research workflow Tablet-first scope Added: cloud + live sharing	2 · UX / UI Design Done User + navigation flows Obsidian Chamber theme Auth / settings / share UI	3 · Architecture Done Shared primitives + stores Per-user storage + session gate Pluggable realtime layer
4 · Implementation Now Core app complete Cloud auth + live share coded Needs keys + native rebuild	5 · Testing / QA Next On-device smoke test Multiplayer dry-run Light / dark / permission cases	6 · Release Next App icons + store assets Crash / analytics opt-in Iterate on demo feedback